

ENTERPRISE KUBERNETES MASTERY

AI Platform Engineering Handbook

PART XII KUBERNETES FOR ENTERPRISE AGENTIC AI

Agent Execution, MCP Servers, A2A, Durable Workflows, Registries, Sovereign AI

Volume 12 of 16 AI Series

Audience: Enterprise architects with existing Agentic AI knowledge

Focus: Kubernetes implementation -- not AI concepts

Edition 2025-2026

TABLE OF CONTENTS

- 1. Kubernetes as the Agentic AI Substrate 3
- 2. Stateless vs Stateful Agent Execution Patterns . 7
- 3. Durable Execution: Temporal on Kubernetes 12
- 4. Argo Workflows for Agent Orchestration 17
- 5. MCP Server Deployment Patterns on Kubernetes .. 22
- 6. A2A Communication Across Clusters 28
- 7. Agent Scheduling and GPU-Aware Placement 33
- 8. Autoscaling Strategies for Agent Workloads 37
- 9. Runtime Isolation for Multi-Tenant Agents 41
- 10. Externalized Memory Services on Kubernetes ... 45
- 11. Tool Registry and Tool Lifecycle Management .. 49
- 12. Prompt Registry and Prompt Lifecycle 53
- 13. Agent Registry and Agent Governance 57
- 14. Workload Identity and Secret Management for Agents . 61
- 15. AI Observability for Multi-Agent Systems 65
- 16. GitOps for AI Assets 69
- 17. Sovereign AI Deployments 73
- 18. Multi-Region Agentic AI Architectures 77
- 19. Disaster Recovery for Agentic Platforms 81
- 20. Hands-On Exercises 84

CHAPTER 1

Kubernetes as the Agentic AI Substrate

This part assumes full knowledge of Agentic AI concepts: LLM-based agents, tool use, memory architectures, RAG, multi-agent orchestration, MCP, A2A, and enterprise AI governance. The focus here is exclusively on Kubernetes implementation patterns: how do you deploy, scale, secure, observe, and govern enterprise Agentic AI systems on Kubernetes?

Why Kubernetes for Agentic AI

- * **Workload heterogeneity:** Agentic systems combine CPU-bound orchestration, GPU-bound inference, stateful memory stores, persistent vector databases, message queues, and event-driven workflows. Kubernetes is the only substrate that manages all of these in a unified operational model with consistent observability and policy.
- * **Dynamic scaling:** Agent workloads are inherently bursty. A single user request may spawn 0 to 50 sub-agent tasks. Kubernetes HPA and KEDA scale the agent worker pool in seconds. Karpenter provisions new GPU nodes in under 2 minutes.
- * **Workload isolation:** Enterprise multi-tenant agent platforms must isolate customer workloads from each other. Kubernetes Namespaces, NetworkPolicies, RuntimeClasses, and RBAC provide layered isolation at compute, network, and credential levels.
- * **Operational consistency:** Agent systems are complex distributed applications. Kubernetes brings the same deployment, rollout, health checking, and observability patterns used for traditional microservices to AI agents -- reducing operational burden and enabling GitOps-based management of the entire platform.
- * **Ecosystem:** The CNCF ecosystem provides: Argo Workflows for orchestration, Temporal for durable execution, LiteLLM for model gateway, Milvus for vector storage, Feast for feature serving, cert-manager for identity, Vault for secrets -- all Kubernetes-native and interoperable.

Enterprise Agentic AI Platform Reference Architecture

```
EXTERNAL INTERFACES API Gateway (Kong/AWS API GW) -> Agent Orchestration API WebSocket
endpoint for streaming agent responses Webhook receivers for event-triggered agents | AI
GATEWAY LAYER (Kubernetes Deployment) LiteLLM / Kong AI Gateway Auth, rate limiting, model
routing, cost tracking | AGENT ORCHESTRATION LAYER Agent API Server (FastAPI/gRPC, Kubernetes
Deployment) Temporal Worker Pool (Kubernetes Deployment, KEDA-scaled) Argo Workflows
Controller | AGENT EXECUTION LAYER Agent Worker Pods (stateless, KEDA-scaled) MCP Server
Deployments (tool-specific, sidecar pattern) GPU Inference Pods (vLLM, KServe) | MEMORY AND
KNOWLEDGE LAYER Short-term: Redis (session memory, conversation history) Long-term:
Qdrant/Milvus (vector memory, knowledge base) Episodic: PostgreSQL (structured agent history,
audit log) Semantic: Feast (feature store, user/entity profiles) | DATA AND MODEL LAYER Model
Registry (MLflow, Kubeflow Model Registry) Tool Registry (custom CRD or Service Catalogue)
Prompt Registry (ConfigMap-based or dedicated service) Object Storage (S3/GCS): datasets,
model weights, artifacts
```

CHAPTER 2

Stateless vs Stateful Agent Execution Patterns

The most consequential architectural decision for an agent execution system is the choice between stateless and stateful execution. This choice determines the scalability model, failure recovery characteristics, cost profile, and operational complexity of the entire platform.

Stateless Agent Execution

In stateless execution, the agent worker Pod holds no local state between steps. All context -- conversation history, intermediate results, tool outputs -- is externalised to a shared state store (Redis, PostgreSQL, object storage). If the Pod crashes at any point, another Pod can resume execution from the last committed state with no data loss.

```
Stateless agent execution pattern: 1. Request arrives at Agent API Server 2. API Server creates task in task queue (Redis/SQS/Kafka) Task includes: session_id, agent_config, initial_message 3. Worker Pod dequeues task 4. Worker loads context from state store (Redis): GET agent:session:{session_id}:context 5. Worker executes agent step: - Call LLM via AI Gateway - Execute tool calls - Update context 6. Worker saves updated context to state store: SET agent:session:{session_id}:context {updated_context} EX 3600 7. If more steps needed: enqueue next step If complete: emit result event Kubernetes resources: Agent API Server: Deployment (stateless REST/gRPC) Task Queue: Redis or Kafka StatefulSet Worker Pool: Deployment, KEDA-scaled on queue depth State Store: Redis StatefulSet + PVC Advantages: Horizontal scale: add workers without coordination Fault tolerance: pod crash = task re-queued from last checkpoint Zero sticky sessions: any worker can process any task Simple autoscaling: KEDA on queue depth
```

Stateful Agent Execution

In stateful execution, the agent process maintains its own in-memory context throughout the lifetime of an agent session. This is simpler to implement but requires sticky session routing (requests for the same session go to the same Pod) and makes horizontal scaling more complex.

```
Stateful agent execution pattern: 1. Request arrives at Agent API Server 2. API Server checks session affinity: GET session:{session_id}:pod_ip from Redis 3. Route to existing Pod (if active) OR assign to new Pod 4. Agent Pod holds full context in memory No serialization overhead; fast step transitions 5. Pod crash = session loss (unless checkpoint mechanism added) Kubernetes resources: Agent Server: StatefulSet with stable Pod identities OR Deployment with session-affinity Service Session Router: sidecar or service-mesh layer Use case: interactive real-time agents where latency matters and session duration is short (< 5 minutes) Avoid for: long-running agents, batch agents, regulated workloads requiring audit trail of every step
```

Decision Matrix

Dimension	Stateless	Stateful
Context storage	External (Redis, PostgreSQL, S3)	In-memory (Pod heap)

Dimension	Stateless	Stateful
Fault tolerance	Excellent (resume from checkpoint)	Poor (Pod crash = session lost)
Horizontal scaling	Simple (add workers)	Complex (sticky routing required)
Latency per step	Higher (state serialization)	Lower (in-memory access)
Session duration	Any length	Best for short sessions
Autoscaling	KEDA on queue depth	HPA on memory/connections
Best for	Batch agents, long-running, regulated	Real-time interactive agents

CHAPTER 3

Durable Execution: Temporal on Kubernetes

Temporal is the leading durable workflow execution engine. It enables writing agent workflows as ordinary code (Go, Java, Python, TypeScript) where every step is automatically checkpointed, retried on failure, and resumable after arbitrary downtime. For enterprise agentic AI, Temporal solves the hardest operational problem: ensuring long-running agent workflows complete reliably even when individual steps fail, infrastructure restarts, or the agent takes hours to complete.

Temporal Architecture on Kubernetes

```
Temporal components: Frontend Service (Deployment): gRPC API for workflow operations (start, signal, query, terminate) Client SDK connects here History Service (Deployment): Stores workflow event history (every step, input, output) Executes deterministic workflow code replay State: backed by PostgreSQL or Cassandra Matching Service (Deployment): Matches tasks to workers via task queues Routes activities (tool calls) to appropriate worker types Worker Service (Deployment, custom code): Runs workflow and activity implementations Scales independently; KEDA-scaled on task queue depth Web UI (Deployment): Visual workflow inspection, debugging, manual intervention Persistence: PostgreSQL (StatefulSet) or managed RDS/CloudSQL Elasticsearch (optional): advanced search over workflow history
```

Temporal Deployment with Helm

```
# Install Temporal on Kubernetes: helm repo add temporal https://go.temporal.io/helm-charts
helm install temporal temporal/temporal \ --namespace temporal --create-namespace \ --set
server.replicaCount=3 \ --set server.config.persistence.default.driver=sql \ --set
server.config.persistence.default.sql.driver=postgres12 \ --set
server.config.persistence.default.sql.host=postgres \ --set
server.config.persistence.default.sql.port=5432 \ --set
server.config.persistence.default.sql.database=temporal \ --set
server.config.persistence.default.sql.user=temporal \ --set
server.config.persistence.default.sql.password=temporal \ --set prometheus.enabled=true \
--set grafana.enabled=true
```

Agentic Workflow in Temporal (Python SDK)

```
import asyncio from datetime import timedelta from temporalio import activity, workflow from
temporalio.client import Client from temporalio.worker import Worker @activity.defn async def
call_llm(prompt: str, model: str) -> str: # Activity: one LLM call. Temporal retries on
failure. response = await llm_client.chat(model=model, prompt=prompt) return response.content
@activity.defn async def execute_tool(tool_name: str, tool_input: dict) -> dict: # Activity:
one tool call. Retried on network failure. result = await tool_registry.call(tool_name,
tool_input) return result @workflow.defn class ResearchAgentWorkflow: @workflow.run async def
run(self, task: str) -> str: # Temporal checkpoints after EVERY await # If pod crashes here,
replay resumes from last checkpoint plan = await workflow.execute_activity( call_llm,
args=[f'Plan how to research: {task}', 'gpt-4o'], start_to_close_timeout=timedelta(minutes=2),
retry_policy=RetryPolicy(maximum_attempts=3) ) # Execute tool calls durably search_results =
await workflow.execute_activity( execute_tool, args=['web_search', {'query': task}],
start_to_close_timeout=timedelta(minutes=1) ) # Synthesize results synthesis = await
workflow.execute_activity( call_llm, args=[f'Synthesize: {search_results}',
```

```
'claude-3-5-sonnet'], start_to_close_timeout=timedelta(minutes=3) ) return synthesis
```


CHAPTER 4

Argo Workflows for Agent Orchestration

Argo Workflows is a Kubernetes-native workflow engine that executes DAGs and step-based workflows as Kubernetes Pods. For agentic AI, Argo Workflows is particularly suited for batch agent pipelines, data processing workflows, and multi-step AI pipelines where each step can run independently.

Argo Workflows vs Temporal

Dimension	Argo Workflows	Temporal
Execution unit	Kubernetes Pod (each step)	Code function (activity)
State persistence	Kubernetes etcd (workflow CRD)	Dedicated database (PostgreSQL)
Step granularity	Container-level (coarse)	Function-level (fine)
Long-running workflows	Good (days, weeks)	Excellent (years if needed)
Sub-second steps	Poor (Pod startup overhead)	Excellent (in-process)
Language	YAML workflow definition	Go, Java, Python, TS SDK
GPU workloads	Native (Pod resources)	Via activity worker with GPU
Best for	ML pipelines, batch AI, data processing	Real-time agents, interactive, fine-grained retry

Agentic Research Pipeline in Argo Workflows

```
apiVersion: argoproj.io/v1alpha1 kind: Workflow metadata: name: research-agent-pipeline namespace: ai-agents spec: entrypoint: research-dag arguments: parameters: - name: research_topic value: enterprise AI adoption trends 2025 templates: - name: research-dag dag: tasks: - name: plan template: llm-call arguments: parameters: - name: prompt value: 'Create a research plan for: {{workflow.parameters.research_topic}}' - name: model value: gpt-4o - name: web-search template: tool-call dependencies: [plan] arguments: parameters: - name: tool value: web_search - name: query value: '{{tasks.plan.outputs.parameters.search_queries}}' - name: synthesize template: llm-call dependencies: [web-search] arguments: parameters: - name: prompt value: 'Synthesize findings: {{tasks.web-search.outputs.result}}' - name: model value: claude-3-5-sonnet-20241022 - name: llm-call inputs: parameters: - name: prompt - name: model container: image: harbor.corp/ai-agent-tools:v2 command: [python, -m, agent_tools.llm_call] args: ['--prompt', '{{inputs.parameters.prompt}}', '--model', '{{inputs.parameters.model}}'] env: - name: AI_GATEWAY_URL value: http://litellm.ai-gateway.svc.cluster.local:4000 resources: requests: { cpu: 500m, memory: 1Gi } securityContext: runAsNonRoot: true runAsUser: 1000 readOnlyRootFilesystem: true
```

CHAPTER 5

MCP Server Deployment Patterns on Kubernetes

Model Context Protocol (MCP) servers expose tools, resources, and prompts to LLM-based agents via a standardised protocol. Deploying MCP servers on Kubernetes enables centralised, scalable, secure tool infrastructure that multiple agents can consume simultaneously.

MCP Server Deployment Topologies

Topology	Description	Isolation	Scalability	Best For
Sidecar per agent	MCP server as sidecar container in agent Pod	High (per-pod)	Limited (scales with agent)	Sensitive tool access; process isolation
Shared Deployment	One MCP server Deployment, many agent clients	Low (shared process)	High (HPA/KEDA)	Stateless tools; high-throughput; cost-efficient
Per-namespace Deployment	MCP server Deployment per team namespace	Medium (namespace)	Medium	Multi-tenant; team tool customisation
Gateway (aggregator)	Single MCP gateway routing to multiple backends	High (gateway)	Very high	Enterprise; unified tool registry; audit logging

Shared MCP Server Deployment

```
# Production MCP server for web search and code execution tools: apiVersion: apps/v1 kind:
Deployment metadata: name: mcp-tools-server namespace: ai-platform spec: replicas: 3 selector:
matchLabels: { app: mcp-tools-server } template: spec: serviceAccountName: mcp-tools-sa
securityContext: runAsNonRoot: true runAsUser: 1000 seccompProfile: { type: RuntimeDefault }
containers: - name: mcp-server image: harbor.corp/mcp-tools:v1.5 command: [python, -m,
mcp_server] args: - --transport=streamable-http - --host=0.0.0.0 - --port=8080 -
--tools=web_search,code_executor,file_reader,database_query env: - name: SEARCH_API_KEY
valueFrom: secretKeyRef: name: mcp-tool-secrets key: search-api-key ports: - name: mcp
containerPort: 8080 resources: requests: { cpu: 500m, memory: 512Mi } limits: { memory: 1Gi }
readinessProbe: httpGet: { path: /health, port: 8080 } periodSeconds: 5 securityContext:
readOnlyRootFilesystem: true allowPrivilegeEscalation: false capabilities: { drop: [ALL] } ---
apiVersion: v1 kind: Service metadata: name: mcp-tools-server namespace: ai-platform spec:
selector: { app: mcp-tools-server } ports: - name: mcp port: 8080 targetPort: 8080 --- # KEDA
ScaledObject: scale MCP server on tool call rate: apiVersion: keda.sh/v1alpha1 kind:
ScaledObject metadata: name: mcp-tools-scaler spec: scaleTargetRef: name: mcp-tools-server
minReplicaCount: 2 maxReplicaCount: 20 triggers: - type: prometheus metadata: query:
sum(rate(mcp_tool_calls_total[1m])) threshold: '50'
```

MCP Server Security Controls

- * **Authentication:** MCP clients (agents) authenticate to the MCP server via Kubernetes Service Account tokens or mTLS. The MCP server validates the identity before executing tool calls.

- * **Authorisation:** RBAC at the MCP level: agent identity determines which tools it can invoke. Use OPA or a custom admission layer in the MCP server to enforce tool-level permissions based on the agent's service account.
- * **Tool sandboxing:** Code execution tools must run in isolated environments. Use gVisor (RuntimeClass kata-gvisor) for the code execution container. Never run code execution in the same Pod as the MCP server process.
- * **Audit logging:** All tool calls logged with: agent identity, tool name, input parameters (sanitised), output (truncated), timestamp, request ID. Essential for EU AI Act compliance and security forensics.
- * **NetworkPolicy:** MCP server should only accept connections from authorised agent namespaces. Default-deny egress except to explicitly allowed external services.

CHAPTER 6

A2A Communication Across Clusters

Agent-to-Agent (A2A) communication in enterprise environments must cross cluster, namespace, and sometimes regional boundaries. Kubernetes provides the networking primitives; the A2A protocol layer sits on top of standard HTTP/gRPC or message queue infrastructure.

A2A Communication Patterns

Pattern	Transport	Kubernetes Implementation	Best For
Synchronous HTTP	HTTP/gRPC	Service + Ingress/Gateway API	Request-response, real-time
Async via message queue	Kafka / NATS / SQS	Strimzi Kafka or NATS Operator	Decoupled, high-throughput
Event-driven	CloudEvents / Kafka	KEDA triggers on events	Reactive agent activation
Shared state	Redis / PostgreSQL	StatefulSet	Shared working memory between agents
Cross-cluster direct	Cilium Cluster Mesh	ClusterMesh service export	Low-latency cross-cluster
Cross-cluster via gateway	HTTP + mTLS	Istio multi-cluster / Linkerd	Federated agent networks

A2A via Kafka on Kubernetes

```
# Strimzi Kafka for agent message bus: apiVersion: kafka.strimzi.io/v1beta2 kind: Kafka
metadata: name: agent-bus namespace: ai-platform spec: kafka: replicas: 3 listeners: - name:
plain port: 9092 type: internal tls: false - name: tls port: 9093 type: internal tls: true
storage: { type: persistent-claim, size: 100Gi, class: fast-ssd } zookeeper: replicas: 3
storage: { type: persistent-claim, size: 10Gi } # Agent topics: # ai-agents.task-requests
(orchestrator -> worker agents) # ai-agents.task-results (worker agents -> orchestrator) #
ai-agents.tool-calls (agents -> MCP servers) # ai-agents.tool-results (MCP servers -> agents)
# ai-agents.events (system-wide agent lifecycle events) # KEDA trigger: scale agent workers on
Kafka lag: triggers: - type: kafka metadata: bootstrapServers:
agent-bus-kafka-bootstrap.ai-platform:9092 consumerGroup: research-agent-workers topic:
ai-agents.task-requests lagThreshold: '10' offsetResetPolicy: latest
```

Cross-Cluster A2A with Cilium Cluster Mesh

```
# Export agent API Service from cluster-1 to cluster-2: # (On cluster-1) Annotate the Service
for global visibility: kubectl annotate service agent-orchestrator \ -n ai-agents \
service.cilium.io/global=true \ service.cilium.io/shared=true # On cluster-2, agents can now
call: # http://agent-orchestrator.ai-agents.svc.cluster.local # Traffic is load-balanced
```

```
across both clusters' endpoints # For isolation (cluster-1 endpoints only preferred): kubectl  
annotate service agent-orchestrator \ service.cilium.io/global=true \  
service.cilium.io/shared=false # Only exported, not imported
```

CHAPTER 7

Agent Scheduling and GPU-Aware Placement

Agent workloads have diverse and often unpredictable resource requirements. A research agent might spawn sub-agents that require different compute profiles: a web scraper (CPU only), a summarisation step (GPU), a code execution sandbox (isolated CPU), and a database query (CPU + network). Kubernetes scheduling must handle this heterogeneity efficiently.

Agent Worker Pod Resource Profiles

Worker Type	CPU	Memory	GPU	RuntimeClass	Use Case
Orchestrator	2-4 cores	4-8 GB	None	runc	Agent planning, routing, coordination
LLM Caller	1-2 cores	2-4 GB	None	runc	LLM API calls, response parsing
Tool Executor	1-2 cores	2-4 GB	None	runc or gVisor	Web search, API calls, file read
Code Executor	2-4 cores	4-8 GB	None	gVisor or Kata	Sandboxed code execution
Local Inference	8-16 cores	64-256 GB	1-4 GPUs	runc	On-premise LLM, embedding
Data Processor	4-16 cores	16-64 GB	Optional	runc	ETL, RAG ingestion, chunking

Node Pool Strategy for Agentic AI

```
# Recommended node pool architecture:
NODE POOL 1: cpu-general Instance: c5.2xlarge (8 CPU, 16 GB) Workloads: orchestrators, LLM callers, tool executors Taints: none (default pool)
NODE POOL 2: cpu-memory-optimised Instance: r5.4xlarge (16 CPU, 128 GB) Workloads: RAG ingestion, embedding generation, data processing Taints: workload-type=memory-intensive:NoSchedule
NODE POOL 3: gpu-inference Instance: g5.12xlarge (48 CPU, 192 GB, 4x A10G) Workloads: local LLM inference, embedding models Taints: nvidia.com/gpu=true:NoSchedule
NODE POOL 4: gpu-training Instance: p4d.24xlarge (96 CPU, 1152 GB, 8x A100) Workloads: fine-tuning, RAG embedding generation at scale Taints: nvidia.com/gpu=true:NoSchedule, workload-type=training:NoSchedule
NODE POOL 5: isolated-execution Instance: c5.xlarge (4 CPU, 8 GB) Workloads: code execution (gVisor), untrusted tool calls Taints: workload-type=isolated:NoSchedule RuntimeClass: gvisor (enforced via Kyverno)
```

CHAPTER 8

Autoscaling Strategies for Agent Workloads

Agent workloads exhibit extreme burstiness that standard CPU/memory-based autoscaling handles poorly. A single customer request can spawn 0 to 100 sub-agent tasks in milliseconds, requiring the platform to scale from 2 to 200 workers in seconds. KEDA event-driven autoscaling is the correct tool for this pattern.

KEDA for Agent Worker Scaling

```
# Scale agent workers based on Kafka topic lag: apiVersion: keda.sh/v1alpha1 kind:
ScaledObject metadata: name: research-agent-workers namespace: ai-agents spec: scaleTargetRef:
apiVersion: apps/v1 kind: Deployment name: research-agent-worker minReplicaCount: 2 #
Always-warm minimum maxReplicaCount: 100 # Peak capacity cooldownPeriod: 120 # 2 min before
scale-down pollingInterval: 5 triggers: - type: kafka metadata: bootstrapServers:
agent-bus-kafka-bootstrap:9092 consumerGroup: research-agents topic: ai-agents.task-requests
lagThreshold: '5' # 1 worker per 5 pending tasks offsetResetPolicy: latest - type: prometheus
metadata: serverAddress: http://prometheus:9090 metricName: agent_active_sessions query:
sum(agent_sessions_active) threshold: '10' # 1 worker per 10 active sessions advanced:
restoreToOriginalReplicaCount: false horizontalPodAutoscalerConfig: behavior: scaleUp:
stabilizationWindowSeconds: 0 # Scale up immediately policies: - type: Percent value: 200 #
Can triple in one step periodSeconds: 15 scaleDown: stabilizationWindowSeconds: 300 # Wait 5
min before scale-down
```

Karpenter for Dynamic GPU Node Provisioning

```
# Karpenter NodePool for GPU agent workers: apiVersion: karpenter.sh/v1 kind: NodePool
metadata: name: gpu-agent-nodes spec: template: metadata: labels: node-type: gpu-agent spec:
nodeClassRef: apiVersion: karpenter.k8s.aws/v1 kind: EC2NodeClass name: gpu-node-class
requirements: - key: karpenter.k8s.aws/instance-family operator: In values: [g5, p3, p4d] -
key: karpenter.sh/capacity-type operator: In values: [spot, on-demand] # Prefer spot for cost
taints: - key: nvidia.com/gpu value: 'true' effect: NoSchedule disruption:
consolidationPolicy: WhenEmpty # Remove idle GPU nodes consolidateAfter: 5m
```

CHAPTER 9

Runtime Isolation for Multi-Tenant Agents

Multi-tenant agentic AI platforms must isolate workloads from different customers or teams at multiple levels: compute, network, credentials, and data. The appropriate isolation level depends on the trust model and regulatory requirements.

Isolation Levels for Agentic AI

Isolation Level	Mechanism	Attack Surface	Overhead	Use Case
Namespace	RBAC + NetworkPolicy + ResourceQuota	Shared kernel	Minimal	Internal teams, trusted tenants
vCluster	Virtual K8s cluster per tenant	Shared kernel (namespaces)	Low	Dev environments, isolated control planes
gVisor (runsc)	User-space kernel per container	Separate user-space kernel	10-20%	Semi-trusted agent code, SaaS
Kata Containers	VM per Pod	Separate VM kernel	3-8%	Untrusted tenant code, regulated
Cluster per tenant	Separate cluster	Fully isolated	Highest	High-value customers, compliance-driven

Kata Containers for Untrusted Agent Code Execution

```
# Enforce Kata for all agent code execution in a namespace: # Kyverno policy: code-exec
namespace must use kata RuntimeClass apiVersion: kyverno.io/v1 kind: ClusterPolicy metadata:
name: require-kata-in-code-exec-ns spec: validationFailureAction: Enforce rules: - name:
require-kata-runtime match: resources: kinds: [Pod] namespaces: [agent-code-execution]
validate: message: Code execution namespace requires Kata Containers runtime pattern: spec:
runtimeClassName: kata-qemu --- # Code execution agent Pod: spec: runtimeClassName: kata-qemu
# VM isolation securityContext: runAsNonRoot: true seccompProfile: { type: RuntimeDefault }
containers: - name: code-executor image: harbor.corp/sandbox-executor:v1 resources: limits: {
cpu: 2, memory: 4Gi } securityContext: readOnlyRootFilesystem: true capabilities: { drop:
[ALL] }
```


CHAPTER 10

Externalized Memory Services on Kubernetes

Agentic AI systems require multiple categories of memory, each with different latency, persistence, and sharing requirements. All memory is externalised from agent Pods into dedicated Kubernetes services, enabling stateless agent execution and horizontal scaling.

Memory Architecture

Memory Type	Content	Latency	Persistence	Kubernetes Service
Working memory	Current task context, conversation turns	< 1ms	Session duration	Redis (in-memory)
Episodic memory	Past agent interactions, action history	< 5ms	Long-term	PostgreSQL + TimescaleDB
Semantic memory	Encoded knowledge, embeddings, facts	< 10ms	Permanent	Qdrant / Milvus / pgvector
Procedural memory	Tool definitions, agent prompts, workflows	< 1ms	Version-controlled	ConfigMap / Prompt Registry
Working set cache	Recently retrieved documents, tool outputs	< 1ms	Short-term	Redis with TTL
Entity memory	User/entity profiles, preferences	< 5ms	Long-term	Feast Feature Store + Redis

Redis for Agent Working Memory

```
# Redis deployment for agent session memory: # Using Bitnami Redis Sentinel for HA: helm
install redis bitnami/redis \ --namespace ai-agents \ --set architecture=replication \ --set
auth.enabled=true \ --set auth.existingSecret=redis-secret \ --set replica.replicaCount=2 \
--set master.persistence.size=50Gi \ --set master.persistence.storageClass=fast-ssd # Agent
working memory patterns: # Session context: agent:session:{id}:context (TTL: 1 hour) # Tool
cache: agent:tool_cache:{hash} (TTL: 5 minutes) # Rate limiting: agent:ratelimit:{user_id}
(TTL: 1 minute) # Active sessions: agent:active_sessions (sorted set by last_seen) # Memory
management: automatically evict LRU on memory pressure maxmemory 50gb maxmemory-policy
allkeys-lru maxmemory-samples 10
```

CHAPTER 11

Tool Registry and Tool Lifecycle Management

A tool registry is the system of record for all tools available to agents. It stores tool definitions, versions, access policies, SLAs, and deployment status. Without a tool registry, tool discovery is ad-hoc, versioning is uncontrolled, and access governance is impossible at enterprise scale.

Tool Registry Design

```
Tool Registry CRD-based approach: apiVersion: ai.company.com/v1alpha1 kind: AgentTool
metadata: name: web-search-v2 namespace: ai-platform labels: tool-category:
information-retrieval tool-version: v2.1.0 data-classification: public annotations:
tool.ai.company.com/owner: platform-team tool.ai.company.com/sla: '99.9'
tool.ai.company.com/max-latency-ms: '2000' spec: description: Search the web for information
using Tavily API mcp_server: service: mcp-tools-server namespace: ai-platform port: 8080
tool_name: web_search input_schema: type: object properties: query: { type: string,
description: Search query } max_results: { type: integer, default: 10 } required: [query]
access_policy: allowed_namespaces: [ai-agents, ai-research, ai-support]
allowed_service_accounts: [research-agent-sa, support-agent-sa] rate_limit:
100/minute/service_account cost_model: cost_per_call: 0.001 # USD billing_dimension: requests
```

Tool Lifecycle Governance

- * **Development:** Tool author registers tool in dev namespace. Integration tests run via Argo Workflows. Security scan: SBOM + CVE check on tool container image.
- * **Staging:** Tool promoted to staging via GitOps PR. Load testing at target RPS. Access policy review by security team.
- * **Production:** Tool available in production tool registry. Backstage catalogue entry created automatically. Prometheus ServiceMonitor deployed for observability.
- * **Deprecation:** New version released; old version marked deprecated. Agents pinned to old version receive upgrade notifications. Traffic migration via canary (Argo Rollouts).
- * **Retirement:** Old version removed from registry. Agent configurations that reference retired tools flagged by admission webhook. Audit trail preserved.

CHAPTER 12

Prompt Registry and Prompt Lifecycle

In enterprise agentic AI, system prompts, few-shot examples, and instruction templates are operational assets that must be versioned, tested, reviewed, and deployed with the same rigour as application code. A prompt registry provides the governance infrastructure for the prompt lifecycle.

Prompt Registry Implementation Options

Approach	Storage	Versioning	Access	Best For
Git + ConfigMap	ConfigMap in cluster	Git history (immutable)	Kubernetes RBAC	Simple; GitOps-native
LangSmith Hub	LangSmith cloud	Semantic versioning	API key	LangChain users
Weights and Biases	W&B; Artifacts	Run-linked versioning	W&B; API	ML-heavy teams
Custom CRD (Prompt resource)	etcd via K8s API	ResourceVersion + labels	RBAC + admission	Enterprise governance
Database + API service	PostgreSQL + REST API	Full semantic versioning	OIDC + RBAC	Largest scale; audit requirements

ConfigMap-Based Prompt Registry (GitOps)

```
# Prompt stored as ConfigMap (versioned via GitOps): apiVersion: v1 kind: ConfigMap metadata:
name: prompt-research-agent-v3 namespace: ai-platform labels: prompt-name: research-agent
prompt-version: v3.2.1 prompt-type: system model-family: gpt-4 language: en annotations:
prompt.ai.company.com/owner: ai-platform-team prompt.ai.company.com/approved-by:
alice@company.com prompt.ai.company.com/approved-at: 2025-06-01T10:00:00Z
prompt.ai.company.com/test-results: passed (100% evals) data: system_prompt: | You are an
enterprise research analyst with access to web search, database query, and document analysis
tools. Always cite sources. Never reveal internal company data. Response format: structured
JSON with citations. few_shot_examples: | [Example 1: Research query -> structured response]
[Example 2: Multi-step research with tool use] # Agent deployment mounts prompt as volume:
volumes: - name: agent-prompts configMap: name: prompt-research-agent-v3 containers: -
volumeMounts: - name: agent-prompts mountPath: /prompts readOnly: true env: - name:
SYSTEM_PROMPT_PATH value: /prompts/system_prompt
```

CHAPTER 13

Agent Registry and Agent Governance

As enterprise agent deployments scale to hundreds of specialised agents, an agent registry becomes essential: a directory of available agents, their capabilities, access policies, resource requirements, and deployment status. The agent registry integrates with Backstage for discoverability and with Kubernetes RBAC for governance.

Agent CRD Design

```
apiVersion: ai.company.com/v1alpha1 kind: Agent metadata: name: research-agent-v3 namespace: ai-agents labels: agent-type: research agent-version: v3.0.0 data-classification: confidential spec: description: Enterprise research agent for market analysis capabilities: - web_search - document_analysis - database_query - report_generation model_config: primary_model: gpt-4o fallback_model: claude-3-5-sonnet embedding_model: text-embedding-3-large prompt_ref: configmap: prompt-research-agent-v3 namespace: ai-platform tool_refs: - name: web-search-v2 namespace: ai-platform - name: database-query-v1 namespace: ai-platform resource_requirements: cpu: 2 memory: 4Gi max_concurrent_tasks: 10 max_context_tokens: 128000 access_policy: allowed_callers: - service_account: research-portal-sa namespace: research-portal - service_account: executive-assistant-sa namespace: executive-tools data_access: allowed_data_sources: [public-web, internal-kb] forbidden_data_sources: [pii-database, financial-records]
```

CHAPTER 14

Workload Identity and Secret Management for Agents

Each agent must have a unique, verifiable identity for accessing external services, calling other agents, and leaving audit trails. Kubernetes Service Accounts combined with SPIFFE/SPIRE or cloud workload identity provide cryptographic agent identity without managing long-lived credentials.

Agent Identity Architecture

```
Agent identity hierarchy: Agent Type Identity (shared by all instances of agent type):
ServiceAccount: research-agent-sa SPIFFE ID:
spiffe://company.com/ns/ai-agents/sa/research-agent-sa Bound token TTL: 1 hour (auto-rotated
by kubelet) Agent Instance Identity (unique to each running instance): Pod name:
research-agent-7d9f8-abc12 Included in audit logs for forensic tracing Session Identity
(unique to each agent session): Generated session_id: uuid4() Propagated in:
X-Agent-Session-ID header, Kafka message key Used for: multi-turn conversation correlation,
cost attribution Cloud Resource Access (via Workload Identity): GKE: research-agent-sa -> GCP
Service Account -> S3/GCS/BigQuery EKS: research-agent-sa -> IAM Role (IRSA) ->
S3/DynamoDB/Bedrock
```

Secret Injection for Agent Tools

```
# External Secrets Operator: sync tool API keys from Vault: apiVersion:
external-secrets.io/v1beta1 kind: ExternalSecret metadata: name: agent-tool-secrets namespace:
ai-agents spec: refreshInterval: 1h secretStoreRef: name: vault-backend kind:
ClusterSecretStore target: name: agent-tool-secrets creationPolicy: Owner data: - secretKey:
tavily-api-key remoteRef: key: secret/ai-agents/tools property: tavily-api-key - secretKey:
anthropic-api-key remoteRef: key: secret/ai-agents/models property: anthropic-api-key # Agent
Pod uses projected volume for secrets: volumes: - name: agent-secrets secret: secretName:
agent-tool-secrets defaultMode: 0400 # Read-only for owner only containers: - volumeMounts: -
name: agent-secrets mountPath: /run/secrets/agent readOnly: true # Never use env vars for
secrets! env: - name: SECRETS_PATH value: /run/secrets/agent
```

CHAPTER 15

AI Observability for Multi-Agent Systems

Observing multi-agent systems requires tracking causality across dozens of LLM calls, tool executions, and sub-agent invocations that compose a single user request. Standard APM tools are insufficient; AI-specific observability must capture model inputs/outputs, tool call chains, token costs, and agent decision points.

Agent Observability Signals

Signal	What to Capture	Tool	Retention
Agent trace	Full span tree: user request -> orchestrator -> sub-agents -> tools -> LLMs	Tempo / Jaeger	30 days
LLM call spans	Model, prompt hash, tokens (in/out), latency, finish reason	OpenTelemetry + Tempo	30 days
Tool call spans	Tool name, input hash, output hash, latency, success/failure	OpenTelemetry + Tempo	30 days
Agent session log	Complete conversation with tool calls (structured JSON)	Loki	90 days
Token usage metrics	Tokens per model per session per agent per user	Prometheus	1 year
Agent cost metrics	USD cost per session, task, agent type, user	OpenCost + custom	1 year
Error and retry events	Failed LLM calls, tool timeouts, retries with context	Prometheus + Loki	90 days
Agent decision log	Which agent was chosen, why, confidence (if available)	Loki structured log	1 year (compliance)

OpenTelemetry for Agent Tracing

```
# Python agent with OTel instrumentation: from opentelemetry import trace from
opentelemetry.sdk.trace import TracerProvider from
opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter tracer =
trace.get_tracer('research-agent', '3.0.0') async def run_agent_task(task: AgentTask) ->
AgentResult: with tracer.start_as_current_span('agent.run') as span:
span.set_attribute('agent.type', 'research') span.set_attribute('agent.session_id',
task.session_id) span.set_attribute('agent.task_id', task.task_id)
span.set_attribute('agent.user_id', task.user_id) with
tracer.start_as_current_span('agent.llm_call') as llm_span:
```

```
llm_span.set_attribute('gen_ai.system', 'openai')
llm_span.set_attribute('gen_ai.request.model', 'gpt-4o') response = await
call_llm(task.prompt) llm_span.set_attribute('gen_ai.usage.prompt_tokens',
response.usage.prompt_tokens) llm_span.set_attribute('gen_ai.usage.completion_tokens',
response.usage.completion_tokens) if response.tool_calls: for tool_call in
response.tool_calls: with tracer.start_as_current_span('agent.tool_call') as tc_span:
tc_span.set_attribute('tool.name', tool_call.function.name) result = await
execute_tool(tool_call) tc_span.set_attribute('tool.success', result.success) return
AgentResult(response=response.content)
```

CHAPTER 16

GitOps for AI Assets

GitOps principles apply to AI assets as much as to application deployments. Prompts, agent configurations, tool registries, model deployments, and pipeline definitions should all be managed via Git with pull-request-based review and automated deployment via ArgoCD or Flux.

AI GitOps Repository Structure

```
ai-gitops/ agents/ research-agent/ agent.yaml # Agent CRD definition deployment.yaml # Worker
Deployment keda-scaler.yaml # KEDA ScaledObject servicemonitor.yaml # Prometheus metrics
prompts/ research-agent/ v3.2.1/ system_prompt.txt # System prompt (Git-versioned)
few_shot.json # Few-shot examples eval_results.json # Test results (must pass before merge)
configmap.yaml # K8s ConfigMap for prompt tools/ web-search/ v2.1.0/ tool.yaml # AgentTool CRD
deployment.yaml # MCP server Deployment tests/ # Tool integration tests models/
llama3-70b-instruct/ v1.0/ inference-service.yaml # KServe InferenceService
vllm-deployment.yaml # vLLM Deployment model-metrics.yaml # ServiceMonitor pipelines/
research-pipeline/ workflow.yaml # Argo Workflow template temporal-workflow.py # Temporal
workflow code config/ ai-gateway/ litellm-config.yaml # Model routing config policies/
agent-rbac.yaml # RBAC for agent service accounts
```

Prompt CI/CD Pipeline

```
# .github/workflows/prompt-release.yaml # Automated prompt evaluation before deployment: name:
Prompt CI/CD on: pull_request: paths: ['prompts/**'] jobs: evaluate: runs-on: ubuntu-latest
steps: - uses: actions/checkout@v4 - name: Run prompt evaluation suite run: | python -m pytest
prompts/research-agent/v3.2.1/evals/ \ --llm-provider openai \ --min-pass-rate 0.95 \ --output
eval_results.json - name: Check for prompt injection vulnerabilities run: python -m
prompt_security_scan prompts/ - name: Update eval results in repo run: | cp eval_results.json
\ prompts/research-agent/v3.2.1/eval_results.json git add -A && git commit -m 'Update eval
results' deploy: needs: evaluate if: github.ref == 'refs/heads/main' steps: - name: ArgoCD
sync run: argocd app sync prompts-production
```


CHAPTER 17

Sovereign AI Deployments

Sovereign AI refers to deploying AI infrastructure entirely within a specific jurisdiction, with full control over data, models, and compute -- no data leaving the sovereign boundary. This is required for government agencies, regulated financial services, healthcare data, and enterprises operating under data residency requirements.

Sovereign AI Requirements

- * **Data residency:** All data processed by agents must remain within the geographic and legal jurisdiction specified. No data sent to external LLM APIs. No telemetry or logs leaving the boundary.
- * **Model sovereignty:** LLM models deployed on-premise or in sovereign cloud. No dependency on external model providers (OpenAI, Anthropic). Models must be hosted entirely within the sovereign boundary.
- * **Supply chain control:** All software components (container images, dependencies) pulled from internal registries, not public internet. Air-gapped cluster with no outbound internet access.
- * **Cryptographic control:** Encryption keys managed within the sovereign boundary (on-premise HSM or sovereign cloud KMS). No key material accessible by cloud provider or third parties.
- * **Audit and compliance:** Complete audit trail of all agent actions, data access, and model invocations. Stored within the sovereign boundary. Exportable for regulatory inspection.

Air-Gapped Kubernetes for Sovereign AI

```
Air-gapped deployment architecture: INTERNET-CONNECTED DMZ ZONE: CI/CD system (scrapes
approved packages from internet) Artifact repository (stores approved, scanned images) |
[ONE-WAY DATA DIODE / TRANSFER MECHANISM] | SOVEREIGN (AIR-GAPPED) ZONE: Internal Harbor
registry (receives images from DMZ) Kubernetes cluster (pulls only from internal registry) All
LLM models stored on internal NFS/Ceph Internal HSM for key management Internal OIDC provider
(no external identity) All observability data stays in cluster # containerd mirror config for
air-gapped cluster: [plugins.'io.containerd.grpc.v1.cri'.registry.mirrors]
[plugins.'io.containerd.grpc.v1.cri'.registry.mirrors.'docker.io'] endpoint =
['https://harbor.internal.sovereign:443/v2/dockerhub-mirror']
[plugins.'io.containerd.grpc.v1.cri'.registry.mirrors.'quay.io'] endpoint =
['https://harbor.internal.sovereign:443/v2/quay-mirror']
[plugins.'io.containerd.grpc.v1.cri'.registry.mirrors.'registry.k8s.io'] endpoint =
['https://harbor.internal.sovereign:443/v2/k8s-mirror'] # Kyverno policy: no external registry
images allowed: spec: validationFailureAction: Enforce rules: - name: only-internal-registry
validate: message: Only harbor.internal.sovereign images allowed pattern: spec: containers: -
image: harbor.internal.sovereign/*
```

CHAPTER 18

Multi-Region Agentic AI Architectures

Enterprise agentic AI platforms serving global users require multi-region deployment to meet latency requirements, data residency regulations, and availability SLAs. The architecture must balance consistency (shared knowledge base, model versions) with locality (data residency, low latency).

Multi-Region Architecture Patterns

Pattern	Data Locality	Latency	Complexity	Best For
Active-Active	Replicated across regions	Lowest (local routing)	Highest	Global SaaS, no strict residency
Active-Passive	Primary region; failover	Medium (failover)	Medium	HA without global latency
Region-per-jurisdiction	Strict per-region isolation	Medium	Medium	GDPR, data residency laws
Hub-spoke	Central hub; regional spokes	Medium	Medium-high	Central governance, regional execution

Hub-Spoke Agent Architecture

```
Hub-spoke pattern for regulated enterprises: HUB CLUSTER (us-east-1, primary): Agent Registry (central directory of all agents) Prompt Registry (canonical prompt versions) Tool Registry (canonical tool definitions) MLflow Model Registry (central model catalogue) ArgoCD (manages all spoke clusters via GitOps) Vault (secret distribution to spoke clusters) SPOKE CLUSTER EU (eu-west-1, GDPR boundary): Agent Workers (process EU user requests) LLM Inference (local, EU-resident models) Redis (EU user session memory, EU-resident only) Vector DB (EU knowledge base, EU-resident only) Data: never leaves EU boundary SPOKE CLUSTER APAC (ap-southeast-1): Agent Workers LLM Inference Redis + Vector DB (APAC-resident data) Cross-cluster communication: Hub -> Spokes: GitOps (ArgoCD ApplicationSets) Hub -> Spokes: Vault secret sync (Enterprise replication) Spoke -> Hub: Metrics (Thanos multi-cluster query) Spoke -> Hub: Logs (Loki multi-tenant, region-labelled) Spoke -> Spoke: NOT ALLOWED (no cross-boundary data flow)
```

CHAPTER 19

Disaster Recovery for Agentic Platforms

Agentic AI platforms introduce unique DR challenges: in-flight agent workflows that may be mid-execution, accumulated vector database indexes, trained model weights, and complex multi-service state. A DR strategy must address each component with appropriate RPO and RTO targets.

Component-Level DR Strategy

Component	DR Strategy	RPO	RTO	Implementation
Kubernetes cluster	Velero backup + standby cluster	1 hour	30 min	Velero daily + Velero pre-change
etcd (cluster state)	Snapshot to S3 every 30 min	30 min	15 min	Automated etcd backup CronJob
Agent workflows (Temporal)	Temporal persistence replicated (Cassandra multi-DC)	Near-zero	Minutes	Temporal HA with multi-region DB
Agent workflows (Argo)	Workflow CRDs in etcd; etcd backup	1 hour	30 min	Velero includes etcd backup
Redis (session memory)	Redis replication + Velero RDB snapshot	15 min	10 min	Redis Sentinel HA + backup
PostgreSQL (agent history)	Streaming replication + pgBackRest	5 min	15 min	CloudNativePG HA + backup
Vector database	Qdrant backup to S3; rebuild from source	1 hour	1-2 hours	Qdrant snapshot + rebuild
Model weights	Object store (S3/GCS) as primary + cross-region replication	Near-zero	Minutes (re-pull)	S3 CRR; models never in cluster only
Prompt/Tool registry	Git (inherently versioned) + ConfigMap backup	Near-zero	Minutes	ArgoCD re-sync to DR cluster

CHAPTER 20

Hands-On Exercises

Exercise 12.1 -- Deploy a Stateless Agent with KEDA Scaling

Build a minimal stateless agent worker with Redis queue and KEDA autoscaling:

```
# 1. Deploy Redis for task queue and session memory: helm install redis bitnami/redis \ --set
auth.enabled=false \ --set architecture=standalone \ --namespace ai-agents --create-namespace
# 2. Install KEDA: helm install keda keda/keda --namespace keda --create-namespace # 3.
Deploy a simple agent worker Deployment: kubectl apply -n ai-agents -f - <<'YAML' apiVersion:
apps/v1 kind: Deployment metadata: name: demo-agent-worker spec: replicas: 1 selector:
matchLabels: { app: demo-agent-worker } template: metadata: labels: { app: demo-agent-worker }
spec: containers: - name: worker image: python:3.12-slim command: [python, -c] args: - |
import redis, time r = redis.Redis(host='redis-master', port=6379) while True: task =
r.blpop('agent:tasks', timeout=5) if task: print(f'Processing: {task[1]}') time.sleep(2) #
Simulate work resources: requests: { cpu: 100m, memory: 128Mi } YAML # 4. Create KEDA
ScaledObject for Redis list: kubectl apply -n ai-agents -f - <<'YAML' apiVersion:
keda.sh/v1alpha1 kind: ScaledObject metadata: name: agent-worker-scaler spec: scaleTargetRef:
name: demo-agent-worker minReplicaCount: 0 maxReplicaCount: 10 triggers: - type: redis
metadata: address: redis-master.ai-agents:6379 listName: agent:tasks listLength: '5' YAML # 5.
Push tasks and observe scaling: kubectl exec -n ai-agents deploy/redis-master -- \ redis-cli
RPush agent:tasks task1 task2 task3 task4 task5 task6 kubectl get pods -n ai-agents -w # Watch
workers scale up
```

Exercise 12.2 -- Deploy MCP Server and Test Tool Call

Deploy an MCP server and verify tool invocation from an agent client:

```
# Deploy a minimal MCP server with a mock tool: kubectl apply -n ai-agents -f - <<'YAML'
apiVersion: apps/v1 kind: Deployment metadata: name: mcp-demo-server spec: replicas: 1
selector: matchLabels: { app: mcp-demo } template: metadata: labels: { app: mcp-demo } spec:
containers: - name: mcp image: python:3.12-slim command: [pip, install, mcp, -q, '&&', python,
-c] # Run a minimal MCP server with a simple tool resources: requests: { cpu: 100m, memory:
256Mi } ports: - containerPort: 8080 YAML kubectl expose deployment mcp-demo-server
--port=8080 -n ai-agents # Test tool listing via MCP protocol: kubectl exec -n ai-agents
deploy/mcp-demo-server -- \ curl -s http://localhost:8080/tools/list # For production MCP: use
official MCP Python SDK # pip install 'mcp[cli]' # mcp install server.py
```

End of Part XII -- Continue to Parts XIII, XIV, XV, XVI

Remaining volumes cover: Part XIII (Emerging Standards: MCP, A2A, AI Control Planes), Part XIV (Enterprise Reference Architectures for AI and Agentic Platforms), Part XV (Hands-On Implementation Labs: Production Cluster to Full AI Platform), and Part XVI (Future Outlook: GPU-Native Kubernetes, Agent Meshes, Autonomous Platform Operations, Edge AI, Federated AI, AI FinOps, Green AI Infrastructure).